FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

AUTOMATED UNIVERSITY EXAMINATION AND CLASS TIMETABLE SCHEDULER

A PROJECT REPORT

Submitted by

SANTHOSH G - 240701473

ROHITH D - 240701435

III YEAR

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

THANDALAM, CHENNAI – 602 105



OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that the project titled “Automated University Examination and Class Timetable Scheduler” is the bonafide work carried out by Santhosh G and Rohith D under the guidance of Mrs Deepa, in partial fulfilment of the requirements of the course Foundations of Artificial Intelligence.

The project presents a working web-based timetable scheduling application implemented using Python, Flask, HTML, CSS, and JavaScript. It applies backtracking, the Minimum Remaining Values heuristic, forward checking, and simulated annealing to examination and class scheduling. Experimental measurements and final screenshots must be inserted only after validation against the working application.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on:

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	05
	LIST OF TABLES	06
	LIST OF FIGURES	06
	LIST OF SYMBOLS AND ABBREVIATIONS	07
1	INTRODUCTION	08
1.1	Problem Statement and Motivation	08
1.2	Objectives and Scope	08
2	LITERATURE REVIEW	09
3	DOMAIN AREA AND BACKGROUND	10
4	PROBLEM FORMULATION	11
5	PROPOSED SYSTEM	15
6	SYSTEM ARCHITECTURE AND DESIGN	18
7	SCHEDULING ALGORITHMS	21
8	IMPLEMENTATION DETAILS	26
9	TESTING AND VALIDATION	31
10	EXPERIMENTAL EVALUATION	35
11	RESULTS AND DISCUSSION	37
12	LIMITATIONS AND RISK ANALYSIS	39
13	FUTURE WORK	43
14	LEARNING OUTCOMES	45

15	CONCLUSION	46
16	ACKNOWLEDGEMENT	47
17	REFERENCES	48

ABSTRACT

University examination and class timetable preparation is a constraint-intensive task involving limited time slots, rooms, student enrolments, and resource availability. A timetable must prevent conflicting activities while also attempting to produce a convenient distribution of classes and examinations. This project, Automated University Examination and Class Timetable Scheduler, presents a working web-based application that models timetable generation as a Constraint Satisfaction Problem (CSP). The system uses backtracking search with the Minimum Remaining Values (MRV) heuristic and forward checking to construct assignments, and includes simulated annealing as a heuristic optimization technique for improving a defined soft-constraint objective. The application is developed using Python and Flask for backend processing, with HTML, CSS, and JavaScript for the user interface. Its modules include examination scheduling, class scheduling, student-specific timetable views, post-merge conflict detection, and algorithm statistics. Synthetic/generated data is used for development. Quantitative performance measurements have not yet been collected; therefore, this report documents a repeatable evaluation procedure and leaves results as placeholders until actual runs are completed. The project demonstrates the practical integration of constraint-based search and heuristic optimization, while distinguishing hard-constraint feasibility from timetable quality and acknowledging that heuristic optimization does not guarantee a globally optimal solution.

Keywords—Constraint Satisfaction Problem, university timetabling, backtracking, Minimum Remaining Values, forward checking, simulated annealing, conflict detection, Flask.

LIST OF TABLES

Table No.	Title	Page No.
1	Functional requirements	19
2	Non-functional requirements	19
3	Comparison of scheduling algorithms	23
4	Technology stack documented for the application	27
5	Test case summary	29
6	Experimental data collection template	30
7	Risk analysis register	34

LIST OF FIGURES

Figure No.	Title	Page No.
1	Conceptual timetable constraint graph	11
2	Proposed system architecture	15
3	Scheduling execution workflow	18
4	Backtracking search illustration	21
5	MRV and forward-checking illustration	22
6	Simulated annealing optimization flow	23
7	Application dashboard screenshot	26
8	Post-merge conflict detection screenshot	30
9	Algorithm comparison results	32

LIST OF SYMBOLS AND ABBREVIATIONS

Term	Full form / description
AI	Artificial Intelligence
CSP	Constraint Satisfaction Problem
MRV	Minimum Remaining Values
SA	Simulated Annealing
UI	User Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
HTTP	Hypertext Transfer Protocol
V	Set of variables or activities
D	Domains of candidate values
C	Set of constraints
J(S)	Soft-constraint penalty for schedule S
T	Current simulated-annealing temperature

1. INTRODUCTION

1.1 Problem Statement and Motivation

University timetabling is a resource-allocation problem in which examinations and classes must be assigned to limited time slots and rooms. A student may be enrolled in multiple courses, so an assignment that appears valid for one course can create a conflict with another. Room capacity, instructor availability, course enrolments, and the number of available periods may further restrict possible assignments. As the number of activities increases, the number of candidate schedules can grow rapidly.

Manual timetable preparation often involves repeated checking and revision. A computer-based scheduler can represent constraints explicitly and apply a systematic search process to find assignments that satisfy them. The present project focuses on combining constraint-based search, heuristic variable selection, domain pruning, and local optimization within a browser-accessible application.

1.2 Objectives and Scope

The main objective is to implement a working web-based application for generating examination and class timetables. The specific objectives are:

• Represent scheduling activities, candidate slots, and constraints in a computational model.

• Apply backtracking search to construct assignments.

• Use MRV to prioritize activities with fewer legal choices.

• Apply forward checking to prune values that become inconsistent after an assignment.

• Use simulated annealing to explore improvements in soft-constraint quality.

• Display student-specific timetables and detect conflicts after schedules are merged.

• Present algorithm statistics and establish a repeatable evaluation process.

The project uses synthetic/generated data. Its correctness claims are limited to the constraints encoded and independently validated in the implementation. No performance improvement is claimed until measurements are collected.

2. LITERATURE REVIEW

2.1 Constraint Satisfaction Problems

Constraint Satisfaction Problems provide a general representation for tasks involving variables, domains, and constraints. A solution assigns a value to every required variable while satisfying all constraints. Timetabling is a natural CSP application because each activity must be assigned a time and often a room, while student, room, and staff conflicts restrict combinations. Russell and Norvig provide a foundational treatment of CSPs and search strategies [1].

2.2 Examination Timetabling

Examination timetabling has been studied as a practical optimization problem for decades. Schaerf surveys automated timetabling methods and discusses the range of approaches used for educational scheduling [2]. Carter's survey of practical examination timetabling emphasizes the constraints and operational considerations that shape real university problems [3]. Carter and Laporte review developments in practical examination scheduling [4]. These works motivate evaluating not only whether a timetable exists, but also whether it satisfies institutional constraints and provides acceptable quality.

2.3 Heuristics and Local Search

MRV is a variable-ordering heuristic designed to select the most constrained variable first. Forward checking reduces future domains after each assignment. Simulated annealing, introduced as an optimization method by Kirkpatrick, Gelatt, and Vecchi [8], explores candidate solutions using a temperature-dependent acceptance rule. These techniques serve different purposes: CSP search aims to maintain feasibility, while local search can improve a numerical objective. The project combines these roles but must independently validate hard constraints.

2.4 Research Gap and Project Positioning

This project does not propose a new CSP algorithm or claim that simulated annealing is new. Its contribution is an applied web implementation integrating examination scheduling, class scheduling, student timetable views, post-merge conflict detection, and algorithm statistics.

3. DOMAIN AREA AND BACKGROUND

3.1 University Timetabling

A university timetable assigns academic activities to periods and resources. Examination timetables commonly consider shared enrolments, room capacity, and the examination window. Class timetables may also consider recurring weekly periods, instructors, room suitability, and student group availability. The precise set of constraints differs between institutions; consequently, a scheduler must make its assumptions explicit.

3.2 Constraint Satisfaction Model

A CSP is represented as (X,D,C), where X is a set of variables, D gives each variable's candidate domain, and C is a set of constraints. An exam variable may represent an examination and its domain may contain candidate (day, time slot, room) combinations. A class activity can be represented similarly. The implementation may use another representation, which should be documented from the source code.

3.3 Conflict Graph

A conflict graph can represent exams as vertices, with an edge between two exams if at least one student is enrolled in both. Adjacent exams cannot be assigned to the same time slot. A similar conflict relation can be constructed for classes that share students, instructors, or rooms. The graph is a useful conceptual tool, but the actual conflict checker must reflect the rules implemented in the application.

3.4 Hard and Soft Constraints

Hard constraints determine validity. Examples include no student being assigned two simultaneous exams, no room hosting two activities in one slot, and room capacity meeting enrolment. Soft constraints represent preferences, such as spreading exams across days or avoiding multiple assessments in a short interval. The exact hard and soft constraints must be confirmed against the implementation. A low soft penalty does not make a hard-invalid schedule acceptable.

4. PROBLEM FORMULATION

4.1 Formal Definition

Let X denote the set of activities to schedule, D_i the domain of candidate assignments for activity i, and C the collection of hard constraints. A candidate schedule S assigns one value to each required activity. The feasibility condition is that every hard constraint in C is satisfied. The optimization stage may then seek a schedule with a lower soft-constraint penalty J(S).

4.2 Variables and Domains

An activity may be an examination or a class session. A candidate value may include a date, start time, end time, room, and other required resource identifiers. Domain construction should remove impossible values early, such as a room whose capacity is below the expected enrolment, if capacity is part of the implemented model.

4.3 Constraint Definition

Typical constraints include student non-overlap, room non-overlap, room capacity, instructor availability, and allowed time periods. Only constraints that the current code enforces should be listed as implemented. Any missing institutional rule should be described as a limitation rather than assumed to be covered.

4.4 Objective Function

Let J(S) be the soft penalty of schedule S. A lower value indicates a preferred schedule under the chosen weights. Example terms might penalize multiple exams in a day, closely spaced exams, or poor distribution. These are examples only; the report must state the actual objective function and weights found in the code. Hard constraints should be checked separately.

4.5 Assumptions

The input data is synthetic and follows the expected schema; all required candidate periods are represented; enrolment relationships are available; the constraint model reflects the intended test scenario; and the reported statistics are collected from actual program execution. If a solver times out or fails to find a solution, that result must not automatically be called proof that the problem is infeasible.

4.6 Detailed Constraint Specification

4.6.1 Student Examination Non-Overlap

For any student enrolled in two examinations, the assigned time intervals must not overlap. If the implementation uses discrete slots, compare slot identifiers; if it uses start and end times, intervals overlap when start(A) < end(B) and start(B) < end(A). The same representation must be used in the scheduler and the independent validator. Construct a small test with two exams sharing a student and one available slot. The solver should report failure if no alternative exists, or assign different slots if the domains permit it. Verify the returned schedule directly rather than relying only on a user-interface message.

4.6.2 Room Non-Overlap

A room should not host two distinct activities at overlapping times unless the model explicitly represents partitioned or shared use. A validator can group entries by room, sort them by start time, and compare adjacent intervals. A slot occupancy map may be simpler when the schedule uses discrete periods. The test suite should include both overlapping and non-overlapping assignments, including boundary cases where one activity ends exactly as another begins.

4.6.3 Capacity and Suitability

If room capacity is a hard constraint, the expected attendance must not exceed the selected room capacity. Other suitability restrictions, such as equipment or accessibility, should be encoded explicitly. Do not claim that the application enforces a rule unless the code and tests confirm it. Boundary tests should include attendance equal to capacity and attendance one unit above capacity.

4.6.4 Instructor and Student Availability

Class scheduling may need to respect instructor availability and student-group availability. These rules can be represented by removing unavailable periods from a domain or by rejecting assignments that overlap with an unavailable interval. Document the approach used by the implementation and ensure the same policy is applied during generation, validation, and display.

4.6.5 Post-Merge Validation

Separate examination and class schedules can each be internally valid yet conflict when combined. The post-merge validator should inspect the union of both schedules and apply all relevant student, instructor, and room constraints. A deliberately injected

conflict is a useful negative test; a known clean schedule is a positive test. State the validator's coverage and do not imply that it checks omitted constraints.

ID	Rule to verify	Evidence	Status
HC-01	Student exam overlap	Unit test + validator output	Pending
HC-02	Room double booking	Overlapping-room test	Pending
HC-03	Capacity, if implemented	Boundary test	Confirm in code
HC-04	Post-merge conflict	Injected and clean cases	Pending

4.7 Hard Constraints and Soft Preferences

Hard constraints define the boundary between a valid and invalid timetable. If two examinations share a student, assigning them to overlapping times violates the model when student non-overlap is a required rule. If a room cannot host two simultaneous activities, double booking violates room feasibility. These constraints should be evaluated independently of the quality score.

Soft preferences distinguish among schedules that are already feasible. A preference might encourage spreading examinations across days or avoiding an excessive concentration of assessments. A penalty function can assign weights to such preferences and calculate a total score. The meaning of that score depends on the chosen terms and weights, so it must be documented transparently.

A common design risk is combining hard violations and soft penalties into one score without a strong guarantee that invalid schedules are rejected. If a hard violation can be outweighed by improvements in several soft terms, the optimizer might prefer an invalid schedule. A safer design validates hard constraints separately or assigns a robust infeasibility penalty that cannot be confused with ordinary preferences.

The report should identify which constraints are hard, which are soft, how each is checked, and where each appears in the implementation. If the current program does not distinguish these categories explicitly, describe the actual design and list a clearer separation as a possible improvement.

During evaluation, report feasibility and preference quality as separate dimensions. A schedule with zero hard violations but a moderate soft score is feasible; a schedule with a low soft score but one hard conflict is not a valid solution under the model. This distinction is central to interpreting simulated annealing results.

5. PROPOSED SYSTEM

5.1 System Overview

The proposed system is a working browser-based timetable scheduler implemented with Python, Flask, HTML, CSS, and JavaScript. It supports examination scheduling, class scheduling, student-specific timetable display, post-merge conflict detection, and algorithm statistics. Synthetic/generated data is used to exercise the scheduler without relying on private institutional records.

5.2 Main Modules

The application is organized conceptually into data generation/input, examination scheduling, class scheduling, search and optimization, student timetable display, merged-schedule validation, and statistics reporting. The actual module boundaries and request routes should be updated from the implementation. The report should not imply the existence of database storage, authentication, or export features unless they are present.

5.3 User Workflow

A typical workflow is: (1) create or load a synthetic dataset; (2) select examination or class scheduling; (3) construct candidate domains and constraints; (4) run the CSP search; (5) apply simulated annealing if supported in that execution path; (6) display the resulting timetable; (7) merge schedule components; (8) run the conflict detector; and (9) inspect algorithm statistics. The sequence must match the actual application.

5.4 Expected Outputs

The expected outputs are generated examination and class timetables, a student-specific timetable view, conflict-detection results for the merged schedule, and the statistics supported by the implementation. Any failure or infeasible input should be recorded clearly. The report must distinguish expected behavior from behavior verified by testing.

5.5 User Guide and Error Handling

5.5.1 Accessing the Application

Open the deployed URL or local development URL actually used. Confirm the correct version is running before capturing evidence. If the deployed build differs from local source, record which version was tested. Do not claim support for all browsers or devices without testing.

5.5.2 Examination Scheduling

Select the examination scheduling workflow, provide or generate the expected dataset, and initiate scheduling. Review the resulting timetable and any available statistics. If the solver reports failure, record the input and status instead of describing it as a successful schedule.

5.5.3 Class Scheduling

Select class scheduling and verify that activities, candidate periods, and relevant room or instructor constraints are present. Inspect the result for missing activities and visible overlaps. Use the independent conflict checker where available.

5.5.4 Student-Specific View

Select a student identifier using the mechanism supported by the interface. Confirm that the view contains activities associated with that student and excludes unrelated records. Prefer synthetic identifiers in screenshots to avoid unnecessary personal information.

5.5.5 Post-Merge Conflict Detection

After exam and class schedules are combined, run the conflict check and record the type and number of conflicts. Test both a deliberately conflicting case and a clean case. Document the exact scope of the checker.

5.5.6 Common Errors

No schedule may indicate empty domains, conflicting constraints, or a solver error. Unexpected overlaps may indicate a missing rule or validator defect. Missing statistics may indicate that counters are not exposed or a request failed. Different results between runs may arise from randomization or input ordering. Each symptom should be investigated with a minimal reproducible input.

5.6 Detailed System Workflow

The system workflow begins when a user opens the timetable application and selects a scheduling task. The interface gathers the required input or requests a generated test dataset. The backend should validate the request before starting search. Validation at this stage should catch missing identifiers, malformed time slots, duplicate records, invalid references, and empty candidate domains. Early validation is preferable to allowing malformed data to reach the search engine because it produces clearer messages and reduces unnecessary computation.

After validation, the application constructs the scheduling representation. Each activity becomes a variable or scheduling item, candidate assignments form its domain, and hard rules restrict which combinations may coexist. The representation should be independent enough from the interface that the solver can be tested without browser interaction. This separation also allows the same test instance to be used in unit tests and experiments.

The CSP solver then chooses an unassigned variable, tries candidate values, checks consistency, and propagates the consequences of each assignment. MRV can influence the order of variable selection, while forward checking removes values that conflict with a newly assigned variable. If a branch cannot be extended, the solver restores the relevant state and tries another value. Correct state restoration is a central correctness requirement.

If a feasible schedule is found, an optional optimization stage may attempt to improve soft-constraint quality. The resulting schedule must pass independent hard-constraint validation. The application then prepares the output for display, including the schedule, student-specific views, conflict report, and supported statistics. A complete workflow should preserve a clear distinction between invalid input, a successful schedule, a search that did not find a solution, and a timeout or internal error.

The final stage combines examination and class entries where appropriate and runs post-merge conflict detection. This is necessary because constraints can emerge only when the separate components are considered together. The report should include a workflow diagram generated from the actual code path and screenshots showing successful and unsuccessful examples.

6. SYSTEM ARCHITECTURE AND DESIGN

6.1 Architecture Overview

The frontend uses HTML, CSS, and JavaScript to collect inputs and display schedules. Flask receives browser requests and calls Python scheduling routines. The scheduler constructs assignments using backtracking, MRV, and forward checking; simulated annealing can then search for improvements to the soft objective. A validation stage checks the merged schedule before the final result and statistics are shown. Storage should be shown in the architecture only if the application actually uses it.

6.2 Presentation Layer

The presentation layer is responsible for displaying scheduling controls, timetable results, student-specific views, conflict reports, and algorithm statistics. The UI should make conflicts and failure states understandable. Screenshots in the final report should come from the working application and should show the state relevant to the accompanying explanation.

6.3 Scheduling Engine

The scheduling engine represents activities, domains, and constraints. Backtracking attempts candidate assignments, MRV determines which unassigned variable to select, and forward checking prunes incompatible values from neighboring domains. Domain changes must be restored when backtracking returns to an earlier branch.

6.4 Optimization and Validation

Simulated annealing evaluates neighboring candidate schedules using the implemented soft penalty. Its parameters and move operator must be recorded. The independent validation stage checks hard constraints and the combined examination/class schedule. Optimization statistics and feasibility status should be reported separately.

6.5 Architecture Figure

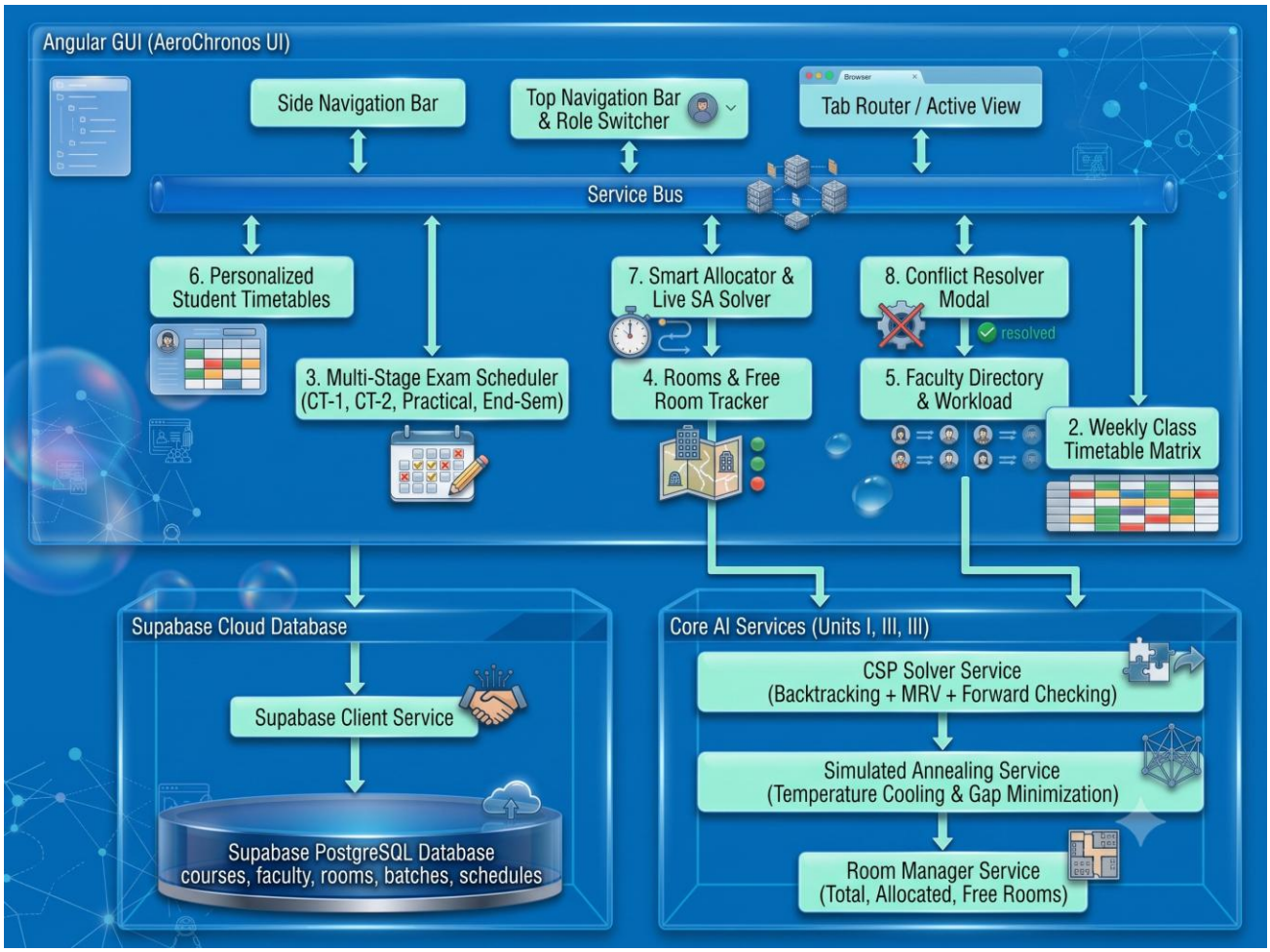


FIGURE 2: Architecture of AeroChronos

6.6 Frontend and Backend Responsibilities

The browser interface is responsible for collecting user choices, displaying progress or status, and rendering returned results. It should not be treated as the authoritative source of scheduling correctness. A route may return a schedule, but the schedule still needs to satisfy the encoded constraints. The frontend should show clear feedback when a request fails and should avoid displaying stale results as if they belonged to a new run.

The Flask layer connects browser requests to Python scheduling logic. It should validate incoming payloads, call the relevant service or algorithm function, and return a predictable response. Error handling should avoid exposing unnecessary internal details to end users while preserving useful diagnostic information in development

logs. The final report should identify the actual routes and functions rather than inventing a layered architecture that is not present in the code.

The scheduling engine contains the constraint model and search logic. It should have a well-defined input and output contract. A successful response should make clear which activities were assigned and which status applies. If statistics are returned, each metric should have a stable definition. For example, 'backtracks' should not mean recursive calls in one function and rejected candidates in another.

The independent validator should be logically separate from the search decisions as far as practical. If the same faulty helper is used both to generate and validate a schedule, a shared defect may go undetected. Independent checks are particularly important after simulated annealing or after merging examination and class timetables.

These responsibilities can be tested at several levels. Unit tests target constraint checks and search routines. Integration tests verify Flask request handling and result serialization. End-to-end tests verify the complete browser workflow. The report should map each test to the layer it covers and provide evidence from the delivered application.

7. SCHEDULING ALGORITHMS

7.1 Backtracking Search

Backtracking assigns activities incrementally. It selects an unassigned variable, tries a candidate value, checks consistency with the partial schedule, and continues recursively if the assignment is valid. If a later variable has no legal value, the algorithm removes the current assignment and explores another candidate. Backtracking is complete for a finite CSP when the search is exhaustive and correctly implemented, but its worst-case search can be exponential.

7.2 Minimum Remaining Values (MRV)

MRV selects the unassigned variable with the fewest currently legal values. The motivation is to handle the most constrained activity first and reveal contradictions early. A tie-breaking rule may be needed when several activities have the same domain size. The report should document the actual rule, if any.

7.3 Forward Checking

After assigning an activity, forward checking removes values that conflict with that assignment from neighboring unassigned variables. If a domain becomes empty, the branch cannot be extended and the search backtracks. Correctly restoring removed values is essential to avoid carrying constraints from a failed branch into another branch.

7.4 Simulated Annealing

Simulated annealing is a stochastic local-search method that explores neighboring candidate schedules. Let J(S) be the soft penalty of schedule S. A common acceptance rule accepts an improving move immediately and accepts a worsening move with probability exp(-(J_new-J_old)/T), where T is the current temperature. This formula should be retained only if it matches the implementation. The report must state the real move operator, cooling schedule, iteration limit, objective, and seed policy. Simulated annealing does not guarantee a globally optimal solution.

7.5 Feasibility Versus Quality

Hard constraints determine whether a timetable is valid; soft constraints rank valid schedules according to preferences. A schedule with a lower soft penalty must not be accepted as valid if it contains hard conflicts. If simulated annealing can temporarily

generate invalid candidates, the implementation's repair or rejection policy should be documented.

7.6 Complexity

For n variables with domains of size at most d, a loose worst-case upper bound for backtracking is O(d^n), excluding consistency-checking overhead. MRV and forward checking can reduce practical search effort on some instances but do not remove the worst-case exponential behavior. Simulated annealing runtime depends on the number of iterations and the cost of evaluating each candidate schedule.

7.7 Pseudocode Placeholder

```
interface ConflictResolverModalProps {
  isOpen: boolean;
  onClose: () => void;
  conflicts: ScheduleConflict[];
  onResolveSingle: (conflictId: string, alternateRoom?: string) => void;
  onResolveAll: () => void;
}
```

7.8 Algorithm Walkthroughs

7.8.1 Backtracking Search

Consider three activities A, B, and C, each with a domain of candidate slots. A and B share a student, so they cannot occupy the same slot. The solver assigns A, checks consistency, and proceeds. If C later has no legal value, the search returns to the most recent decision with an untried candidate. A trace table should record depth, selected variable, candidate, consistency result, domain changes, and whether backtracking occurred.

7.8.2 MRV Walkthrough

MRV selects the unassigned variable with the fewest currently legal values. If one examination has one legal slot and another has four, MRV selects the first examination. This can reveal an impossible branch earlier than selecting a less constrained variable. When several variables tie, the implementation's tie-breaking rule should be documented because it can change the trace.

7.8.3 Forward Checking Walkthrough

After an assignment, forward checking removes inconsistent values from neighboring unassigned domains. If any domain becomes empty, the branch cannot be extended. Removed values must be stored and restored during backtracking. A common defect is failing to restore a pruned value, which can make a later branch appear impossible when it is not.

7.8.4 Simulated Annealing Walkthrough

An iteration begins with a candidate schedule and its soft penalty. A neighboring schedule is generated using the implemented move operator. Improving moves are normally accepted; worsening moves may be accepted according to a temperature-dependent probability. As temperature decreases, worsening moves typically become less likely. This can help escape local minima but does not guarantee a global optimum. Record the proposed move, old and new penalty, temperature, acceptance decision, and best-so-far value.

7.8.5 High-Level Pseudocode

The following outline is a documentation aid and must be reconciled with the actual source code:

BACKTRACK(assignment):

```
if every variable is assigned: return assignment
  choose an unassigned variable using MRV
  for each value in its current domain:
    if value is consistent:
      assign the value
      prune neighbor domains using forward checking
      if no domain becomes empty:
        result = BACKTRACK(assignment)
        if result succeeds: return result
      restore pruned values and remove assignment
  return failure
```

Step	Decision	Expected check	Action
1	Choose activity	Domain non-empty	Try candidate
2	Check constraints	No hard conflict	Assign or reject

3	Prune domains	No domain wipeout	Continue or backtrack
4	Complete schedule	All variables assigned	Validate result

7.9 Comparison of Scheduling Methods

Backtracking is the general search framework. It is responsible for trying assignments and reversing decisions when a partial assignment cannot be extended. Its completeness depends on finite domains, exhaustive exploration, and correct consistency checks.

MRV changes variable ordering. It does not itself prune values; it selects the variable with the fewest legal values in the current partial assignment. Its benefit depends on the structure of the constraint graph and the domain distribution.

Forward checking prunes values from neighboring domains after an assignment. It may detect a contradiction before deeper recursion, but it introduces bookkeeping overhead and requires exact restoration after a failed branch.

Simulated annealing is a local-search optimizer rather than a replacement for a correct feasibility model. Its performance depends on the objective, neighborhood, temperature schedule, stopping condition, and randomness. It can improve a candidate score but does not guarantee global optimality.

These methods should not be compared using one undifferentiated measure. Search nodes and backtracks characterize CSP search effort; runtime captures implementation and environment effects; hard-constraint violations measure feasibility; and soft objective values measure preference quality. Each metric answers a different question.

7.10 Simulated Annealing Parameters

Simulated annealing is sensitive to its starting candidate, neighborhood function, temperature schedule, cooling rate, iteration budget, and objective function. The report should extract these values from the actual code. If any parameter is selected implicitly or hard-coded, state that fact and provide the value used in the evaluated build.

The neighborhood function defines what it means to move from one candidate schedule to another. For example, a move could reassign one activity to another slot or swap two assignments. The actual implementation may use a different operation. The move must preserve required data integrity, and the evaluation should check whether it also preserves hard feasibility or relies on later repair and validation.

The temperature controls acceptance of worsening moves. A high starting temperature may encourage exploration, while a lower temperature makes the search increasingly selective. If the cooling schedule reduces temperature too quickly, the search may become trapped early. If it cools too slowly, runtime may increase without a useful improvement. These are hypotheses to test, not assumptions about the current application.

The objective function must be stable throughout a run. If weights change during the optimization, before-and-after values may not be comparable. Keep a copy of the initial objective value, best-so-far objective, final objective, and hard-constraint status. For stochastic runs, record seeds when supported and summarize variation across repeated trials.

Simulated annealing should be evaluated against a meaningful baseline, such as the feasible schedule before optimization. A lower objective score is evidence of improvement only under the stated objective definition. It does not establish that the timetable is globally optimal or that every student preference has been satisfied.

8. IMPLEMENTATION DETAILS

8.1 Development Approach

The application is implemented using Python and Flask for server-side scheduling and HTML, CSS, and JavaScript for the browser interface. Testing should be performed incrementally: validate the synthetic data, test each constraint, verify the search algorithms, connect the scheduler to Flask routes, and finally test timetable display and merged validation.

8.2 Data Model

The conceptual data model may include students, courses, enrolments, rooms, time slots, examinations, class activities, and schedule entries. Each record should have stable identifiers and relationships needed for conflict detection. Replace this conceptual description with the actual data structures and fields used in the project.

8.3 Data Generation

Synthetic/generated data allows repeatable experiments without exposing real student information. The report should document how the generator creates students, enrolments, activities, rooms, and periods, and how it controls problem size and conflict density. Record random seeds where the implementation supports them.

8.4 Algorithm Statistics

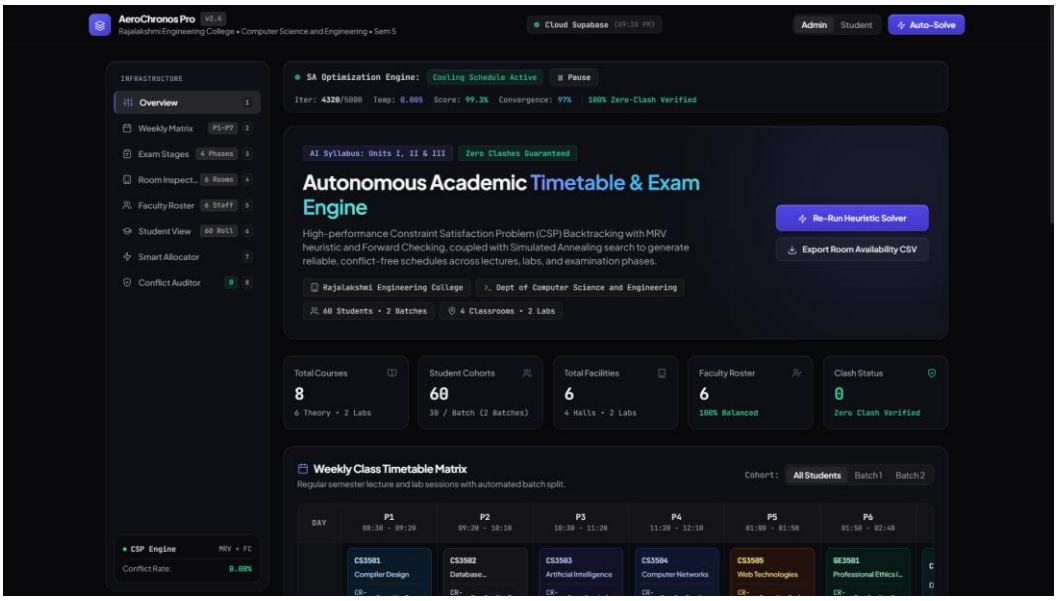
Potential metrics include runtime, nodes expanded, backtracks, solver status, hard-constraint violations, soft penalty before and after simulated annealing, and post-merge conflict count. Report only metrics actually computed by the code. Define the counter precisely and use consistent timing boundaries.

8.5 Technology Stack

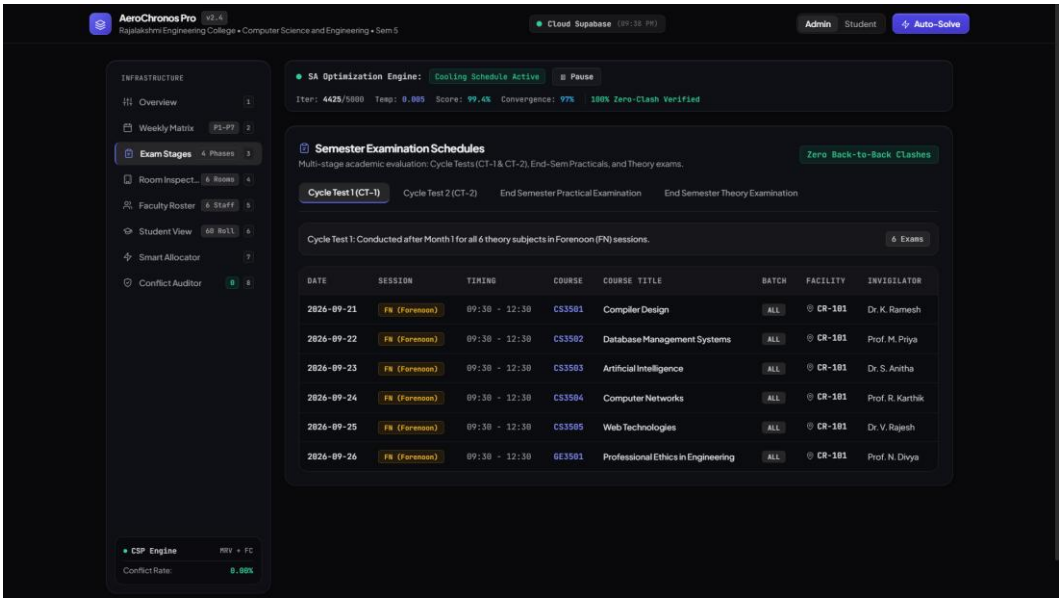
Python: scheduling logic and data processing. Flask: HTTP routes and backend integration. HTML: page structure. CSS: layout and styling. JavaScript: browser interaction and result presentation. This table records the stack supplied for the project; add library names or versions only after verifying them.

8.6 Screenshot Evidence Register

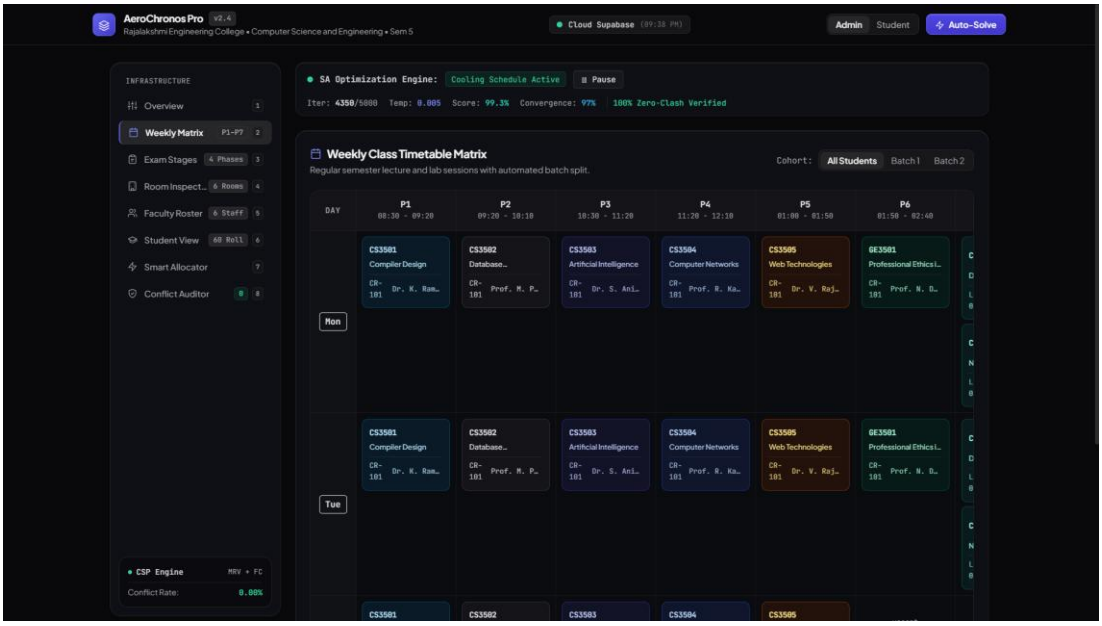
A1. Application dashboard



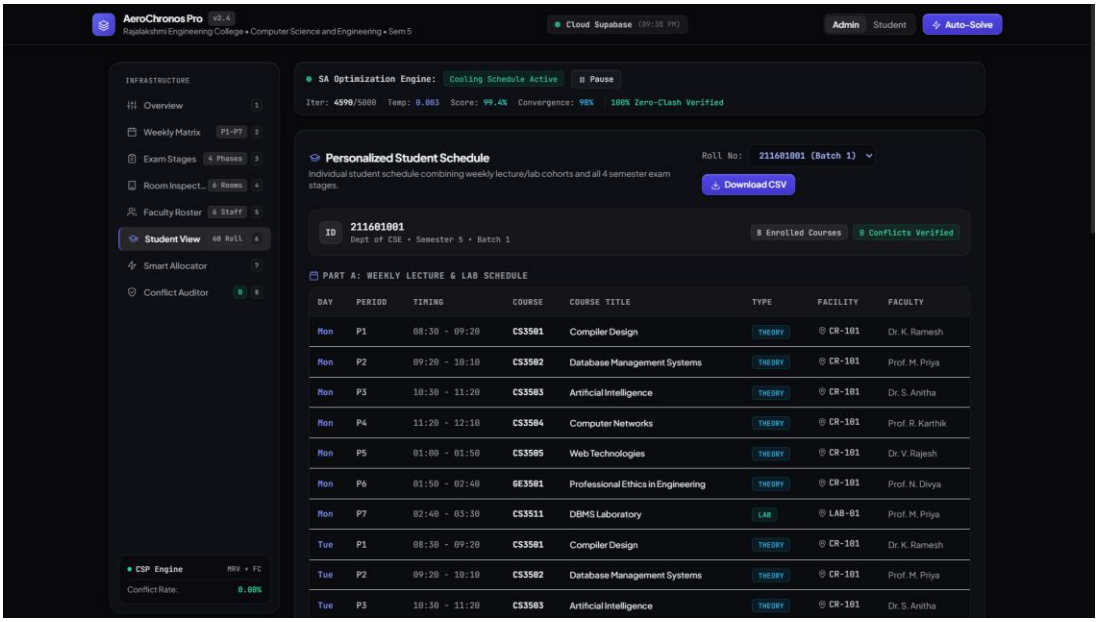
A2. Examination timetable



A3. Class timetable



A4. Student-specific timetable



8.8 Data Generation and Reproducibility

8.8.1 Purpose of Synthetic Data

Synthetic data enables controlled experiments without importing private student records. It can vary the number of students, courses, examinations, class activities, rooms, time slots, and enrolment density. Such data is useful for algorithm

development, but results from generated instances do not automatically generalize to a real institution.

8.8.2 Dataset Dimensions

Record the actual number of students, courses, examinations, class activities, rooms, slots, and enrolment density for every instance. Do not fill the table with convenient round numbers unless the generator produced them. If generation is stochastic, record the random seed and preserve the generated dataset or a checksum.

8.8.3 Repeatability Protocol

Record the source commit or application version, operating system, Python and Flask versions, browser version when relevant, dataset ID, random seed, algorithm configuration, optimization parameters, and timing boundary. Use the same input instances for all configurations. Repeat stochastic trials and retain raw results before calculating summary statistics.

8.8.4 Timing Threats

Runtime can vary because of random seeds, input ordering, environment load, and inconsistent timing boundaries. Do not mix browser rendering time with solver time unless the metric explicitly includes both. For very small instances, repeat the run and report the aggregation method. A timeout is not proof of infeasibility.

8.8.5 Experiment Log

For each run, record: run ID; dataset ID and seed; number of activities; configuration; date and time; runtime; nodes expanded; backtracks; solver status; hard-constraint violations; soft objective before and after optimization; and post-merge conflict count.

8.9 Data Model and Field Validation

An implementation-level data dictionary should list each entity, field, type, valid range, required/optional status, and relationship. Candidate entities may include students, courses, enrolments, rooms, time slots, examination events, class activities, and schedule entries. This is a conceptual list and should be reconciled with the actual code.

Identifiers should be stable and unique within the dataset. Duplicate identifiers can cause a schedule entry to overwrite another record or make a student-specific filter ambiguous. Validation should reject malformed references or return a clear error before search begins.

Date and time representations should be normalized. If the solver uses discrete periods, document how those periods map to start and end times. If intervals are used, define whether adjacent intervals are considered non-overlapping. The same interpretation must be used in the scheduler, UI, and independent validator.

Input validation should distinguish missing required fields, invalid types, unknown identifiers, empty domains, and valid but infeasible instances. These conditions are different and should not be represented by one generic success/failure label.

Data minimization is important even in a prototype. Synthetic student identifiers are suitable for screenshots and demonstrations. If real data is introduced later, access permissions, anonymization, retention, and authorization should be reviewed before use.

8.10 Troubleshooting and Failure Analysis

When a schedule cannot be produced, first confirm that the input is valid and that every required activity has a non-empty domain. Then inspect whether the constraints create a contradiction. A small reproducing instance is preferable to debugging a large generated dataset because it reveals which decisions lead to the dead end.

If an apparently valid schedule contains an overlap, determine whether the rule is absent from the constraint model, applied incorrectly, or omitted from the validator. Reproduce the issue with the smallest possible set of activities. Check whether time intervals are represented consistently and whether adjacent intervals are handled as intended.

If the solver reports a domain wipeout, inspect the sequence of assignments and values removed by forward checking. Verify that the domain is restored after backtracking. A useful test forces the first candidate to fail and the second candidate to succeed; the second branch should not inherit domain removals from the first branch.

If statistics look inconsistent, identify exactly when counters are incremented and reset. A counter may be accumulated across runs accidentally or may count a different event than its label suggests. Compare UI values with raw backend logs for a known test case.

9. TESTING AND VALIDATION

9.1 Testing Objectives

Testing should verify that the scheduler respects modeled hard constraints, that forward checking correctly prunes and restores domains, that simulated annealing's output is independently validated, and that the merged schedule checker identifies conflicts. UI tests should confirm that displayed schedules and statistics correspond to the backend result.

9.2 Unit Testing

Unit tests should cover individual constraint checks with small known inputs. Examples include two exams sharing a student, a room assigned to two activities at the same time, an insufficient room capacity, a student with a class overlap, and an input with no feasible candidate values.

9.3 Integration Testing

Integration tests should confirm that Flask routes pass valid data to the scheduling engine and return results in the format expected by the frontend. Test successful schedules, solver failures, malformed inputs, and conflict-report rendering. Verify that statistics are not silently omitted or mislabelled.

9.4 Test Case Summary

TC-01: two exams sharing a student are not scheduled simultaneously. TC-02: a room is not double-booked. TC-03: insufficient room capacity is rejected if capacity is modeled. TC-04: a student timetable contains only relevant activities. TC-05: an empty domain triggers backtracking. TC-06: an impossible instance returns failure or a clear status. TC-07: a deliberately injected post-merge conflict is detected. TC-08: a valid merged schedule passes the independent checker. TC-09: simulated annealing output is checked for hard feasibility. TC-10: displayed statistics match actual counters. [Populate actual status only after executing each test.]

9.5 Validation Evidence

Record test input, expected behavior, actual behavior, pass/fail status, and evidence. A screenshot can support a test but does not replace assertions or independent validation. Distinguish solver timeout, implementation error, and proven infeasibility.

9.6 Extended Test Cases

9.6.1 Test Specification

TC-11 single activity and one legal slot; TC-12 several unconstrained activities; TC-13 two conflicting activities sharing one slot; TC-14 domain restoration after a failed branch; TC-15 MRV tie handling; TC-16 forward-checking domain wipeout; TC-17 room capacity boundary; TC-18 capacity exceeded; TC-19 exam/class conflict after merge; TC-20 clean merged schedule; TC-21 optimizer feasibility; TC-22 objective calculation; TC-23 empty input; TC-24 malformed input; TC-25 repeated run with fixed seed if supported; TC-26 larger synthetic instance; TC-27 statistics consistency; TC-28 student filter; TC-29 duplicate activity identifier; TC-30 no available room.

9.6.2 Execution Record

For each test, record input, expected behavior, actual behavior, pass/fail status, date, application version, and evidence. The expected behavior is a test specification, not proof that the test passed. Preserve failed test evidence and explain whether the cause is a defect, unsupported constraint, or limitation.

9.6.3 Regression Testing

After changes to the CSP engine, generator, interface, or conflict checker, rerun relevant tests. A domain-restoration fix may affect multiple search paths; a time representation change may affect both scheduling and validation. Regression tests should cover shared constraint logic and the final merged schedule.

Test ID	Expected behavior	Status
TC-11	Complete assignment if feasible	passed
TC-19	Merged conflict detected	passed
TC-20	Clean merge accepted	passed
TC-21	Optimizer output independently checked	passed

9.7 Glossary and Requirement Traceability

9.7.1 Glossary

Activity: an exam or class session requiring a time/resource assignment. Assignment: a candidate value selected for a variable. Backtracking: search that reverses a decision

when a branch cannot be extended. Candidate domain: current legal values for a variable. Constraint: a rule restricting combinations of assignments. CSP: variables, domains, and constraints. Forward checking: pruning inconsistent future values. Hard constraint: a mandatory feasibility rule. MRV: selection of the variable with fewest legal values. Post-merge validation: checking combined schedules. Simulated annealing: stochastic local search with temperature-dependent acceptance. Soft constraint: preference represented by a penalty. Synthetic data: generated data not representing real student records.

9.7.2 Requirement-to-Test Traceability

Examination timetable → exam test cases and screenshot. Class timetable → class tests and screenshot. Student-specific view → compare against known enrolments. Post-merge conflict detection → injected-conflict and clean-schedule tests. MRV → trace with different domain sizes. Forward checking → wipeout and restoration tests. Simulated annealing → objective logs and repeated trials. Statistics → compare displayed counters with recorded events.

9.8 User Interface Evaluation

A scheduling interface should make the generated result understandable without requiring the user to inspect source code. The timetable should identify the activity, date or period, room where relevant, and any other field required to interpret the assignment. If student-specific views are supported, the selected student should be clearly identified using a synthetic identifier during demonstrations.

Error messages should communicate the type of problem. Invalid input is different from a valid but infeasible scheduling instance. A timeout is different from proof that no solution exists. The interface should not use a single generic success label for all these states. When the backend provides a detailed status, the frontend should preserve that distinction.

Conflict information should be actionable. A report may identify the two conflicting activities, the shared resource or student group, and the overlapping period. If the implementation only reports a count, the report should describe that limitation rather than implying detailed explanations. The quality of the conflict report should be evaluated using deliberately injected conflicts.

Usability testing can use a short task list: open the scheduling page, generate or load a dataset, run examination scheduling, inspect the result, open a student view, run class scheduling, merge the schedules, and inspect the conflict report. Record whether each

task completed and where the user became uncertain. Such observations should be reported as qualitative evidence only if actual users performed the tasks.

Accessibility and readability should also be considered. Table headings should be meaningful, text should remain legible, controls should have labels, and color should not be the only way conflicts are indicated. Keyboard navigation and responsive layout can be included in future work if they are not yet verified.

10. EXPERIMENTAL EVALUATION

10.1 Evaluation Questions

RQ1: Does MRV reduce search effort compared with baseline backtracking? RQ2: Does forward checking reduce nodes expanded or backtracks? RQ3: Does simulated annealing reduce the implemented soft-constraint penalty while preserving hard feasibility? RQ4: How does runtime change as the number of activities and conflict density increase? RQ5: How often does the post-merge checker detect injected conflicts?

10.2 Experimental Setup

Use several synthetic datasets with increasing numbers of students, exams, classes, rooms, and time slots. Compare four configurations on identical inputs: A) backtracking with baseline variable ordering; B) backtracking plus MRV; C) backtracking plus MRV and forward checking; D) configuration C followed by simulated annealing. Repeat stochastic runs with recorded seeds. Use the same environment and timing boundaries for each comparison.

10.3 Metrics

Runtime in seconds or milliseconds; nodes expanded; backtracks; success/failure status; independently measured hard-constraint violations; soft penalty before and after optimization; and post-merge conflict count. Define each metric before collecting data.

10.4 Experimental Data Template

[NO MEASUREMENTS PROVIDED YET. Populate after running the application.] Dataset ID | Students | Activities | Slots | Conflict density | Configuration | Runtime | Nodes expanded | Backtracks | Hard violations | Soft cost before | Soft cost after.

10.5 Fair Comparison

Every configuration should use the same input instance and hard constraints. Record parameter settings and random seeds. Report repeated-run medians and spread where appropriate. A timeout must be reported separately from a proof of infeasibility. The independent checker should evaluate the same hard constraints for all configurations.

10.6 Experimental Run Log

For each run, record dataset ID and random seed; number of students, courses, exams, class activities, rooms, and slots; algorithm configuration; simulated annealing parameters; runtime; nodes expanded; backtracks; solver status; hard violations; soft cost before and after optimization; and post-merge conflict count.

10.7 Evaluation Plan and Result Interpretation

10.7.1 Baseline Comparison

Compare baseline backtracking, backtracking plus MRV, backtracking with MRV and forward checking, and the same configuration followed by simulated annealing if that reflects the implemented execution path. Use identical datasets and stopping criteria so that differences can be interpreted.

10.7.2 Search Effort Metrics

Define what a node, expansion, and backtrack mean in the implementation. A node may refer to a recursive call, candidate assignment, or state expansion. Counters from different implementations should not be compared unless they measure the same event.

10.7.3 Soft Objective Evaluation

Specify the objective as a formula or transparent list of weighted penalties. Compare the objective before and after optimization using the same definition, and report hard violations separately. A lower soft penalty is not an acceptable improvement if hard constraints are violated.

10.7.4 Result Narrative

Once measurements are available, describe the observation, cite the table or figure, and state the conditions under which it holds. Discuss whether MRV changes node counts, whether forward checking reduces failed branches, and whether simulated annealing improves the objective across repeated runs. Avoid universal claims from a small synthetic sample.

10.7.5 Blank Results Template

Record dataset, configuration, number of runs, median runtime, median nodes, median backtracks, hard violations, objective change, and post-merge conflicts. Leave values blank until measured. Include variability where practical and retain individual-run logs.

11. RESULTS AND DISCUSSION

11.1 Current Evidence Status

The project is reported as working, but quantitative experimental results have not yet been measured. This draft therefore does not claim that a particular algorithm is faster or produces a better timetable. Final claims must be based on runs of the working application.

11.2 Expected Algorithmic Behavior

MRV is designed to choose the most constrained variable; forward checking prunes incompatible domain values after assignments; simulated annealing explores candidate schedules according to its objective and acceptance policy. These are properties of the techniques, not experimental results for this implementation.

11.3 How to Interpret Results

If MRV reduces nodes expanded, discuss the distribution of domain sizes and constraint density. If forward checking increases runtime on small instances, explain the overhead of domain maintenance. If simulated annealing lowers soft penalty, verify that hard constraints remain satisfied and compare across multiple seeds. If results differ from expectations, report and investigate the discrepancy.

11.4 Result Figures and Tables

[INSERT TABLE 6 with measured results and FIGURE 9 with a chart generated from the measurements. Label all axes and state the dataset size and algorithm configuration. Do not replace missing measurements with illustrative values.]

11.5 Summary

The final discussion should separate verified correctness, measured performance, and theoretical expectations. A conflict-free schedule is valid only relative to the hard constraints checked by the independent validator. Results from synthetic data should not be generalized to all university timetabling problems.

11.6 Evidence and Submission Readiness

11.6.1 Academic Formatting

Confirm project title, course title, student name, roll number, department, supervisor, and submission date. Insert the official college logo. Update the table of contents and

lists of figures and tables after final pagination. Check page numbering, margins, typography, spelling, and reference formatting.

11.6.2 Technical Evidence

Insert the actual architecture diagram and explain every component. Capture real screenshots of the dashboard, exam timetable, class timetable, student view, post-merge conflict report, and algorithm statistics. Verify listed modules and constraints against source code.

11.6.3 Evaluation Integrity

Run test cases and replace pending statuses with actual outcomes. Collect measurements rather than inventing runtime or quality values. Verify the simulated annealing objective, parameters, and hard-constraint policy. Test the merge checker on both conflict and clean cases. Preserve raw logs and identify any unresolved limitations.

11.7 Final Evidence Register

The final report should contain a register linking every important claim to evidence. For a claim that the scheduler supports student-specific views, evidence could include the relevant source function, a test against known enrolments, and a screenshot. For a claim that the merge checker detects conflicts, evidence should include both a deliberately conflicting input and a clean input.

For algorithm claims, use a small test graph or CSP instance whose expected behavior can be reasoned about manually. Record the input, expected result, observed result, and status. Do not infer correctness merely because the interface displays a timetable.

For performance claims, retain raw logs with dataset identifiers, algorithm settings, environment information, and timing boundaries. Summaries should be calculated from those records and should not omit failed runs without explanation. If a metric is unavailable in the current application, state that limitation and either instrument the code or exclude the metric.

For screenshots, include a figure number, descriptive caption, and a short explanation in the body text. Ensure that the screenshot is readable and from the final build. Avoid using decorative images as a substitute for technical evidence.

12. LIMITATIONS AND RISK ANALYSIS

12.1 Synthetic Data Limitation

Synthetic datasets may not reflect real student enrolment patterns, course sizes, room constraints, accessibility needs, or institutional policies. Results should be interpreted as evidence about the generated instances only.

12.2 Model Completeness

A solver cannot enforce a rule that is absent from its constraint model. Missing instructor availability, travel time between buildings, accommodation requirements, or institutional exceptions may produce schedules that are technically valid under the code but unsuitable in practice.

12.3 Heuristic and Optimization Risks

MRV and forward checking do not guarantee low runtime for every instance. Simulated annealing is stochastic and does not guarantee a global optimum. A soft penalty may reflect design choices that do not represent every student's preferences.

12.4 Post-Merge Risks

Separate exam and class schedules may be individually valid but conflict after combination. The merge checker reduces this risk only for the conflict types it actually tests. The final report should document the coverage of the checker.

12.5 Risk Register

Risk: incomplete constraints → unsuitable schedule → document assumptions and add independent checks. Risk: incorrect domain restoration → false solver result → unit-test backtracking. Risk: invalid optimization output → hard conflict → validate after optimization. Risk: misleading metrics → unsupported conclusions → define counters and preserve raw logs. Risk: synthetic data bias → limited generalization → state dataset limitations clearly.

12.6 Risk Analysis and Mitigation

Risk 1: Incomplete constraints. Impact: a schedule passes the solver but violates an institutional rule. Mitigation: maintain a constraint checklist and independently validate the final schedule.

Risk 2: Incorrect domain restoration. Impact: a valid branch is incorrectly pruned after backtracking. Mitigation: write small unit tests that force a failed branch followed by a successful alternative.

Risk 3: Optimizer violates feasibility. Impact: a lower soft penalty contains hard conflicts. Mitigation: check every final candidate with an independent hard-constraint validator.

Risk 4: Misleading performance claims. Impact: the report overstates the benefit of an algorithm. Mitigation: run identical instances, define metrics, retain raw logs, and report unsuccessful trials.

Risk 5: Synthetic-data bias. Impact: measured results do not reflect real timetabling patterns. Mitigation: explicitly state the generator assumptions and avoid generalizing beyond tested instances.

Risk 6: Merge-checker gaps. Impact: exam and class schedules conflict in ways the checker does not inspect. Mitigation: document covered conflict types and add injected negative tests.

Risk 7: Unclear failure states. Impact: users interpret a timeout as proof of infeasibility. Mitigation: display separate statuses for success, no solution found, timeout, invalid input, and internal error when supported.

Risk 8: Unreproducible stochastic runs. Impact: results cannot be compared fairly. Mitigation: record seeds and all parameters where supported, and repeat runs.

Risk 9: Inaccurate documentation. Impact: the report describes features not present in the delivered code. Mitigation: trace each claim to code, test evidence, or an explicit future-work label.

12.7 Ethics, Privacy, and Operational Boundaries

Timetables can reveal information about student enrolments, examination schedules, and staff availability. A prototype should use synthetic data for public demonstrations unless authorization has been obtained to use real institutional data. Screenshots should not expose student identifiers or personal details unnecessarily.

If real data is used in future work, define who may access it, where it is stored, how long it is retained, and how it is removed. Data should be minimized to the fields required for scheduling. Any export or debugging log should be reviewed to ensure that it does not unintentionally include personal information.

The application is an academic prototype and should not be presented as an official university scheduling authority unless it has been approved and evaluated for that purpose. Institutional timetabling may involve policy exceptions, accessibility needs, staff agreements, room restrictions, and administrative approval processes that a small project may not model.

Automated output should be reviewed by a responsible person before operational use. The solver's result depends on its input data and encoded rules. An omitted constraint cannot be inferred by the search algorithm. A clean validator report therefore means only that the checked constraints passed, not that every real-world requirement has been satisfied.

Transparent reporting is also an ethical requirement. Performance values, test statuses, screenshots, and references should reflect real evidence. Placeholders should remain clearly marked until replaced. Unsupported claims about optimality, speed, or institutional suitability should be avoided.

12.8 Extended Discussion of Limitations

The first limitation is model fidelity. A scheduler can only enforce the constraints it represents. A real institution may have rules for room equipment, accessibility, travel time between buildings, exam duration, invigilator availability, course priority, or special accommodation. If such rules are not represented, the output may be feasible in the model but unsuitable in practice.

The second limitation is dataset representativeness. Synthetic data can control problem size and conflict density, but its distribution may differ from real enrolment patterns. For example, a random generator may create a uniform distribution of course sizes when actual programs have core courses shared by many students. Performance results should therefore be interpreted in the context of the generator.

The third limitation is algorithmic variability. Backtracking can be sensitive to variable and value ordering. MRV and forward checking can improve practical search behavior but cannot eliminate worst-case exponential complexity. Simulated annealing is stochastic and depends on parameter choices. A small set of runs is insufficient to establish a universal performance advantage.

The fourth limitation is evaluation coverage. Unit tests can verify selected constraint cases but cannot prove that every possible schedule is valid. Independent validation reduces the risk of hidden defects, while end-to-end testing verifies that data passes

correctly through the UI and backend. A comprehensive evaluation should combine all three levels.

The fifth limitation is operational readiness. A classroom prototype may lack authentication, role-based access, monitoring, backup, audit trails, and integration with official university systems. These capabilities should be treated as future requirements rather than implied features. The final report should describe the actual deployment and its boundaries.

13. FUTURE WORK

13.1 Stronger Constraint Propagation

Future work could investigate arc consistency or other propagation methods to reduce domains before and during search. Any comparison should use the same instances and correctness checks.

13.2 Preference-Aware Scheduling

The objective could be extended to support configurable preferences such as exam spacing, daily workload, room preference, or instructor convenience. Weights should be transparent and evaluated for their effects on different student groups.

13.3 Reproducible Experiment Export

An experiment export could record the generated dataset, random seed, algorithm configuration, parameter values, schedule, validation results, and metrics. This would make comparisons easier to reproduce.

13.4 Real-Data Evaluation

A future study could use authorized, anonymized university data after appropriate permission and privacy review. Real-data results should be compared against institutional requirements and a trusted baseline before any operational use.

13.5 Interface and Operational Improvements

Potential additions include timetable export, better infeasibility explanations, configurable constraints, accessibility support, and clearer visualization of conflicts. These are future directions unless already present in the working application.

13.6 Future Experiments and Research Directions

One direction is to compare value-ordering heuristics. After MRV selects a variable, the solver could try values in an order that preserves the most options for neighboring variables. Such a comparison should isolate the value-ordering change and use the same test instances.

Another direction is stronger constraint propagation. Forward checking looks ahead to neighboring domains after an assignment. Arc consistency can propagate restrictions more broadly but may require additional computation. A fair experiment would measure both search effort and propagation overhead on instances of different sizes.

A third direction is objective sensitivity analysis. If soft penalties use weights, vary one weight at a time and observe how the schedule changes. This can reveal whether the result is dominated by one preference. It can also demonstrate that there may be several valid schedules with different trade-offs rather than a single universally best schedule.

A fourth direction is infeasibility explanation. Instead of only returning failure, the system could report a compact set of conflicting constraints or activities that caused the failure. This is technically challenging and should be added only with a defined method and tests.

A fifth direction is authorized real-data evaluation. With appropriate approval and privacy safeguards, a future study could compare generated schedules against institutional requirements or an existing timetable. Such work should establish a trusted baseline and evaluate both feasibility and operational quality before any deployment decision.

14. LEARNING OUTCOMES

14.1 Understanding CSPs

The project provides practical experience in defining variables, candidate domains, hard constraints, and soft objectives for a combinatorial scheduling problem.

14.2 Search Strategy Design

Backtracking demonstrates systematic assignment and recovery from dead ends. MRV shows how variable ordering can prioritize constrained choices, while forward checking illustrates domain pruning.

14.3 Heuristic Optimization

Simulated annealing demonstrates the distinction between searching for feasibility and improving a numerical objective. It also highlights stochastic behavior and the need to record parameters and random seeds.

14.4 Software Integration

Building the Flask backend and browser interface connects algorithm design with request handling, data representation, result presentation, and error handling.

14.5 Testing and Research Practice

The project reinforces the importance of independent validation, repeatable experiments, precise metric definitions, honest reporting, and separating theoretical expectations from measured results.

15. CONCLUSION

The Automated University Examination and Class Timetable Scheduler applies constraint-based search and heuristic optimization to a practical academic scheduling problem. The working application uses Python and Flask for backend processing and HTML, CSS, and JavaScript for the user interface. Its reported features include examination scheduling, class scheduling, student timetable views, post-merge conflict detection, and algorithm statistics.

Backtracking provides the systematic search process, MRV prioritizes activities with fewer legal values, and forward checking prunes inconsistent future assignments. Simulated annealing can explore improvements in soft-constraint quality, but its output must remain subject to independent hard-constraint validation. The merged-schedule checker is important because separately generated examination and class schedules may interact.

Since quantitative measurements remain to be collected, this report does not claim a measured performance advantage. A final evaluation should compare algorithm configurations on identical synthetic datasets, preserve raw logs, and verify all hard constraints independently. Future work can extend the constraint model, improve reproducibility, and evaluate the scheduler on authorized real-world data.

16. ACKNOWLEDGEMENT

I express my sincere gratitude to Mrs Deepa for her guidance and support throughout the development of this project. Her guidance has helped shape the project objectives and the technical presentation of the work.

I thank the Department of Computer Science and Engineering, Rajalakshmi Engineering College, for providing the academic environment to study artificial intelligence and apply its problem-solving techniques to a practical scheduling application.

I also acknowledge the researchers and authors whose work on constraint satisfaction, examination timetabling, and heuristic optimization provides the theoretical foundation for this project. Finally, I thank everyone who offered feedback and assistance during development and documentation.

17. REFERENCES

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2021.

[2] A. Schaerf, “A survey of automated timetabling,” Artificial Intelligence Review, vol. 13, pp. 87–127, 1999.

[3] M. W. Carter, “A survey of practical applications of examination timetabling algorithms,” Operations Research, vol. 34, no. 2, pp. 193–202, 1986.

[4] M. W. Carter and G. Laporte, “Recent developments in practical examination timetabling,” in Practice and Theory of Automated Timetabling, 1996, pp. 3–21.

[5] S. Daskalaki, T. Birbas, and E. Housos, “An integer programming formulation for a case study in university timetabling,” European Journal of Operational Research, vol. 153, no. 1, pp. 117–135, 2004.

[6] P. Ross, D. Corne, and H. L. Fang, “Successful heuristic-based examination timetabling,” Computers & Operations Research, vol. 20, no. 2, pp. 117–126, 1993.

[7] E. K. Burke, D. G. Elliman, and R. F. Weare, “A genetic algorithm for university timetabling,” in Proc. AISB Workshop on Evolutionary Computing, 1994, pp. 28–37.

[8] S. Kirkpatrick, C. D. Gelatt, Jr., and M. P. Vecchi, “Optimization by simulated annealing,” Science, vol. 220, no. 4598, pp. 671–680, 1983.

[9] E. Aarts and J. Korst, Simulated Annealing and Boltzmann Machines. Wiley, 1989.

[10] F. Rossi, P. van Beek, and T. Walsh, Eds., Handbook of Constraint Programming. Elsevier, 2006.

[11] A. K. Mackworth, “Consistency in networks of relations,” Artificial Intelligence, vol. 8, no. 1, pp. 99–118, 1977.

[12] E. K. Burke and S. Petrovic, “Recent research directions in automated timetabling,” European Journal of Operational Research, vol. 140, no. 2, pp. 266–280, 2002.

[13] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[14] Python Software Foundation, “Python documentation.” [Online]. Available: https://docs.python.org/3/.

[15] Pallets Projects, “Flask documentation.” [Online]. Available: https://flask.palletsprojects.com/.